

Simulación de Eventos Discretos: Algoritmos de Planificación de Procesos

Jabel Resendiz Aguirre

Abril 2025

Resumen

1. Introducción al Planificador de la CPU

El planificador de la CPU es una parte fundamental del sistema operativo de un sistema de computo ya que se encarga de decidir qué proceso va a utilizar la CPU en un momento determinado.

El trabajo del planificador es seleccionar, de entre los procesos que están esperando para ejecutarse, cuál va a ser el siguiente en utilizar la CPU. Para hacerlo, el planificador sigue un conjunto de reglas, que se conocen como algoritmos de planificación.

El planificador tiene un impacto directo en el rendimiento de la computadora. Su tarea no es sólo garantizar que todos los procesos se ejecuten, sino también hacerlo de manera que el sistema funcione de manera eficiente. Esto significa que el planificador debe minimizar el tiempo que los procesos pasan esperando y debe asegurarse de que los recursos de la CPU se utilicen de la mejor manera posible.

En este proyecto de simulación, nos enfocaremos en modelar el comportamiento de un sistema de planificación de CPU utilizando eventos discretos. La idea principal es analizar cómo los algoritmos de planificación, como el Round Robin, Multilevel Feedback Queue y SRTF impactan la ejecución de los procesos en un sistema con una única CPU. Para ello, simularemos cómo

los procesos llegan al sistema y cómo son asignados a la CPU para su ejecución. Utilizaremos distribuciones probabilísticas, como la distribución exponencial y de Poisson, para modelar el tiempo entre llegadas de los procesos y sus tiempos de ejecución, lo que nos permitirá simular un entorno realista donde los tiempos de llegada y duración no son constantes, sino aleatorios.

1.1. Características del sistema

Para describir adecuadamente un sistema de colas, se deben tener en cuenta las siguientes características fundamentales:

- **Patrón de llegada de los procesos:** Los procesos llegan al sistema siguiendo una distribución exponencial, lo que modela un proceso de Poisson. Esto representa un flujo aleatorio y continuo de solicitudes hacia la CPU, adecuado para entornos con alta incertidumbre en los tiempos de llegada.
- **Patrón de servicio del procesador:** El tiempo de CPU requerido por cada proceso se genera aleatoriamente usando distribuciones probabilísticas como la exponencial. Esto permite reflejar la variabilidad natural en la duración de las tareas reales.
- **Disciplina de cola:** Indica el orden en que los clientes son atendidos. En este proyecto se emplean reglas de secuenciación con prioridades, utilizando una técnica de planificación con desalojo (preemptive), en la cual el proceso que se encuentra en ejecución puede ser interrumpido para dar paso a otro de mayor prioridad dentro del sistema. Los algoritmos usados son:
 - **Round Robin (RR):** Un algoritmo de planificación basado en asignar una cantidad fija de tiempo a cada proceso de manera cíclica.
 - **Shortest Remaining Time First (SRTF):** Un algoritmo de planificación preemptivo donde el proceso con el tiempo de ejecución restante más corto se ejecuta primero.
 - **Multilevel Feedback Queue (MLFQ):** Un algoritmo basado en múltiples colas con diferentes prioridades, donde los procesos son promovidos o degradados entre las colas según su comportamiento.

- **Capacidad del sistema:** Es el número máximo de clientes que el sistema puede albergar, incluyendo los que están siendo atendidos y los que están esperando. Si la capacidad se supera, los nuevos clientes pueden ser rechazados o descartados. En nuestro sistema no estableceremos una cota para la cantidad de clientes (procesos), aunque eso no represente un escenario en sistemas embebidos o críticos donde sí hay un límite físico (RAM, threads u otros recursos propios).
- **Número de canales de servicio:** Se refiere al número de servidores disponibles en paralelo. El modelo considera un sistema monoprocesador, donde existe un único canal de servicio (la CPU), encargado de ejecutar los procesos uno a uno según una disciplina de planificación preemptiva
- **Número de etapas de servicio:** En nuestro caso, simulamos un sistema de planificación de CPU en el que cada proceso requiere únicamente una etapa de servicio: la ejecución en la CPU. No modelamos otras fases como entrada/salida (I/O) o espera por recursos externos. Por tanto, se trata de un sistema con una sola etapa de servicio.

1.2. Objetivo y metas

El objetivo principal de este proyecto es comprender y analizar el comportamiento de distintos algoritmos de planificación de CPU a través de una simulación estocástica de un sistema de colas. Además nos permite comparar la eficiencia de distintos algoritmos de planificación de CPU evaluando métricas. Mediante la simulación y el análisis de estos algoritmos, buscamos entender mejor cómo las decisiones de planificación afectan el rendimiento global del sistema.

1.3. Variables de interés

- **pid:** Identificador único para cada proceso. Se utiliza para hacer referencia y distinguir los diferentes procesos que entran al sistema.
- **arrival_time:** Es el tiempo en el que un proceso llega al sistema, determinado por una distribución de llegadas.

- **burst_time:** Tiempo de ejecución necesario para que el proceso complete su tarea. Esta variable es relevante para determinar cuánto tiempo se mantendrá el proceso ejecutándose en el CPU.
- **start_time:** Es el momento en que un proceso comienza su ejecución en el procesador. Esta variable se establece más tarde, cuando el proceso es seleccionado por el planificador.
- **end_time:** Es el tiempo cuando el proceso finaliza su ejecución. Al igual que el tiempo de inicio, este valor se calcula una vez que el proceso ha completado su tarea.
- **waiting_time:** Es el tiempo que un proceso pasa esperando antes de que comience su ejecución. Se calcula como la diferencia entre el tiempo de vuelta y el tiempo de ejecución ($\text{waiting_time} = \text{turnaround_time} - \text{burst_time}$).
- **turnaround_time:** Es el tiempo total que transcurre desde la llegada de un proceso hasta su finalización. Se calcula como la diferencia entre el tiempo de finalización y el tiempo de llegada ($\text{turnaround_time} = \text{end_time} - \text{arrival_time}$).
- **remaining_time:** Indica el tiempo restante para completar la ejecución del proceso. Esta variable es especialmente útil en algoritmos pre-emptivos.

1.4. Variables que describen el problema

Como bien describimos en las características de nuestro sistema, nuestras variables que describen el problema son :

- **Patrón de llegada de los procesos:** El tiempo entre la llegada de los procesos se modela utilizando una distribución *exponencial*, lo cual implica un proceso de *Poisson*. Esto significa que los procesos llegan de manera aleatoria, y el tiempo entre llegadas sigue una distribución exponencial con una tasa promedio de llegada.
- **Patrón de servicio de los procesos:** El tiempo de ejecución de cada proceso (o tiempo de ráfaga) también se modela utilizando una distribución *exponencial*. Este tiempo de servicio determina cuánto tiempo cada proceso ocupará la CPU antes de completarse.

- **Número de procesos:** Este es el número total de procesos que el sistema debe manejar durante la simulación. Los procesos llegarán al sistema según el patrón de llegada descrito previamente.
- **Capacidad de la cola:** Este es el número máximo de procesos que pueden esperar en la cola de ejecución. En este proyecto no se establece una capacidad máxima para la cola, lo que permite que el sistema acepte y procese una cantidad arbitraria de procesos a lo largo de la simulación.
- **Número de canales de servicio:** El sistema tiene un solo canal de servicio, lo que significa que se utiliza un solo procesador de CPU para ejecutar los procesos. Este sistema se considera mono-servidor.
- **Disciplina de cola:** En este proyecto se implementan algoritmos de planificación de CPU como Round Robin, Multilevel Feedback Queue MLFQ o Shortest Remaining Time First (SRTF).

2. Detalles de Implementación

La simulación de planificación de procesos en CPU fue desarrollada en Python mediante un enfoque de eventos discretos. La arquitectura es modular y extensible, permitiendo simular múltiples algoritmos. A continuación se detallan los elementos principales:

- **Estructuras principales:** Se definieron las clases `Process` y `Event` para representar procesos y eventos respectivamente.

Listing 1: Clase Process

```

1      class Process:
2      def __init__(self, pid, arrival, duration):
3          self.pid = pid
4          class Process:
5          def __init__(self, pid, arrival_time,
6              burst_time):
7              self.pid = pid                # Unique
              identifier for each process
8              self.arrival_time = arrival_time    # Time
              when the process arrives
              self.burst_time = burst_time        #
              Execution time of the process

```

```

9         self.start_time = None                # Time
           when the process starts executing (to be
           set later)
10         ...

```

Listing 2: Clase Event

```

1     class Event:
2         def __init__(self, time, event_type, process)
           :
3             self.time = time                    #
           Time when this event occurs
4             self.event_type = event_type        #
           Type of the event (e.g., 'arrival', '
           completion')
5             self.process = process              # The
           process involved in the event
6
7         def __lt__(self, other):
8             """Defines how to compare events"""
9             return self.time < other.time

```

- **Interfaz para los planificadores:** Se definió una interfaz `IScheduling`, que estandariza los métodos que todo algoritmo de planificación debe implementar, como el manejo de llegadas, finalización y ejecución de procesos. Esto facilita la extensibilidad del simulador.

Listing 3: Interfaz `IScheduling`

```

1     class IScheduling(ABC):
2         @abstractmethod
3         def add_process(self, process): pass
4
5         @abstractmethod
6         def process_event(self, event): pass

```

- **Cola de eventos:** Se utilizó una *priority queue* (cola de prioridad) para almacenar eventos, ordenados por tiempo de llegada, es la forma que tiene el algoritmo de saber el orden en que llegan los procesos.

Listing 4: Cola de eventos con `heapq`

```

1     import heapq
2

```

```

3         def add_process(self, process: Process):
4             self.processes.append(process)
5             arrival_event = Event(process.arrival_time, '
6                 ARRIVAL', process)
7             heapq.heappush(self.event_queue, arrival_event)

```

- **Algoritmos de planificación:** Cada algoritmo está encapsulado en una clase que implementa una interfaz común. Se implementaron FCFS, SRTF, Round Robin y MLFQ.

Listing 5: Ejemplo de Round Robin

```

1     class RoundRobin(IScheduling):
2     def __init__(self, quantum):
3         super().__init__()
4         self.quantum = quantum
5         self.ready_queue = deque()
6         self.running_process = None

```

- **Preempción:** En los algoritmos como SRTF o MLFQ, se implementa lógica para interrumpir el proceso actual si uno de mayor prioridad llega. Para ellos se usan colas con prioridad para ir manteniendo la prioridad entre los procesos que ya han sido generados hasta ese momento.
- **Cálculo de métricas:** Se calcularon métricas como el tiempo promedio de espera y de retorno al finalizar todos los procesos.

Listing 6: Tiempo de espera promedio

```

1     def statistics(self):
2         results = {}
3         for scheduler_name, scheduler in [
4             ("MLFQ", self.mlfq_scheduler),
5             ("RoundRobin", self.round_robin_scheduler),
6             ("SRTF", self.srtf_scheduler)
7         ]:
8             turnaround = [p.turnaround_time for p in
9                 scheduler.completed_processes]
10            waiting = [p.waiting_time for p in
11                scheduler.completed_processes]
12            results[scheduler_name] = {
13                'avg_turnaround_time': np.mean(
14                    turnaround) if turnaround else
15                    0,

```

```

12         'avg_waiting_time': np.mean(
13             waiting) if waiting else 0,
14         'processes': len(scheduler.
15             completed_processes)
16     }
17     return results

```

- **Pruebas y validación:** Se ejecutaron pruebas con parámetros aleatorios, distribuciones controladas y se compararon resultados entre algoritmos.

2.1. Funcionamiento de mi Simulación

:

- Se definen mis variables del problema utilizadas para calcular los tiempos de arribo y el tiempo que necesita la CPU para completar cada proceso. Así como también se sabe el tiempo de estudio de la simulación.
- Se procesan los datos de todos los procesos de mi simulación teniendo en cuenta que en nuestra definición los procesos son independientes.
- Se ejecutan los algoritmos definidos previamente ,uno a la vez con la misma lista de procesos previos definidos
- Se recoge y recopila información del estado y la vida de cada proceso en un algoritmo y se muestra , listo para ser procesados
- Se interpretan los datos obtenidos apoyándonos de gráficos y tablas.
- Se ejecutan pruebas con parámetros aleatorios, distribuciones controladas y se comparan resultados entre algoritmos.

3. Resultados y Experimentos

En la simulación, los algoritmos de planificación fueron evaluados con un conjunto de procesos predefinidos. La salida mostrará el tiempo de finalización de cada proceso, lo que permitirá analizar el rendimiento de cada algoritmo.

3.1. Hipótesis y Experimentos

Se realizaron varios experimentos para evaluar cómo cada algoritmo maneja los procesos con diferentes tiempos de ejecución (burst times). Los resultados se compararon en términos de tiempo total de ejecución y tiempo de espera promedio.

4. Conclusiones

Los resultados mostraron que el algoritmo **Round Robin** tiende a ser más eficiente cuando todos los procesos tienen tiempos de ejecución similares, mientras que el **Shortest Remaining Time First** es más eficiente cuando hay grandes variaciones en los tiempos de ejecución de los procesos. El **Multilevel Feedback Queue** es útil cuando se requiere un control dinámico de la prioridad de los procesos.